VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

Bhopal-Indore Highway, Kothrikalan, Sehore

Madhya Pradesh - 466114

CSE3006 – Data Mining & Warehousing

REG.NO : 23BAI10917

NAME : VIJAY RAJESH R

BRANCH : AIML

SEMESTER: WINTER SEMESTER 2025-26

INDEX

Exp.No	Date	Experiment Name	Page No
1.	10-01-2026	Exploring WEKA and Building an Anonymous Data Warehouse	03
2.	29-01-2026	Implementation of several data pre-processing tasks on datasets.	08
3.	06-02-2026	Implementation of association rule mining on data sets.	12
4.	13-02-2026	Implementation of classification techniques: Naïve Baye's, and SVM on data sets.	18
5.	28-03-2026	Implementation of regression techniques: linear, logistics and Neural Networks on data sets.	24
6.	28-03-2026	Implementation of k-means and graph-based clustering techniques on data sets.	30
7.	29-03-2026	Credit Risk Assessment. Sample Programs using German Credit Data.	34
8.	04-03-2026	Sample Programs using Hospital Management System.	37
9.	04-03-2026	Simple Project on Data Pre-processing.	41

EXP.NO: 01	Exploring WEKA and Building an Anonymous Data Warehouse
DATE: 10-01-2026	

AIM

To Explore WEKA and Building an Anonymous Data Warehouse using a custom dataset.

PROCEDURE

1. Launch EC2 Instance: Use the AWS Console to launch a t2.micro instance with Amazon Linux 2023 and an appropriate Key Pair.

2. Configure Security Group: Ensure the instance's Security Group allows inbound traffic on Port 22 (SSH) and Port 80 (HTTP).

3. Connect via SSH: Use your downloaded .pem key file to SSH into the instance using the public IP address.

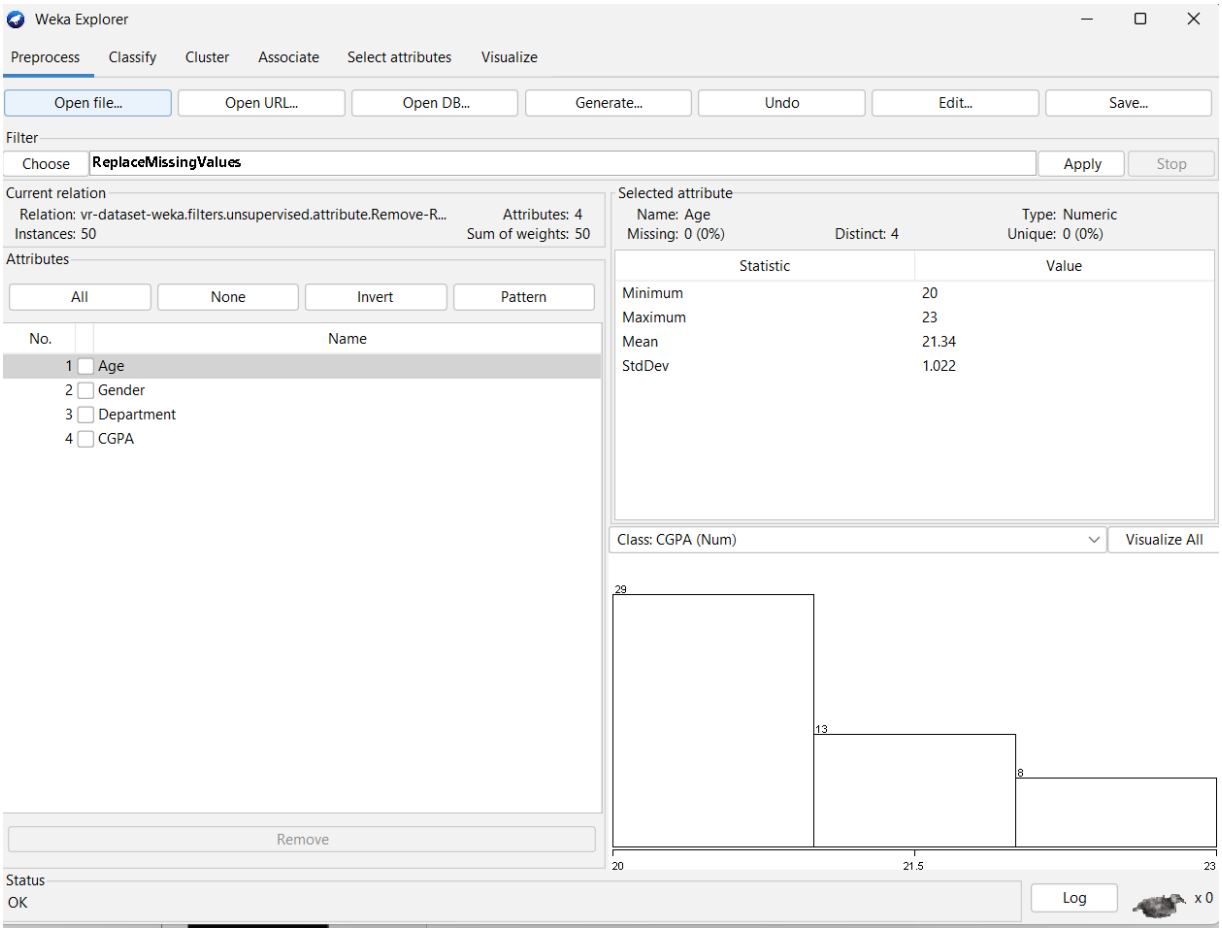
4. Install Apache: Run sudo dnf update -y then sudo dnf install httpd y on the instance.

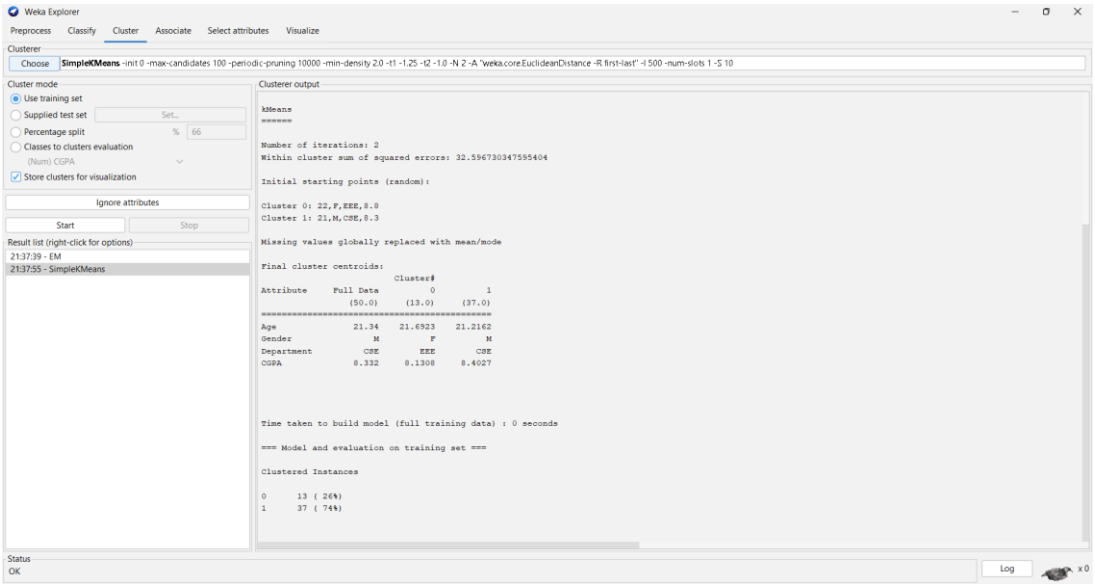
5. Start and Enable Service: Run sudo systemctl start httpd and sudo systemctl enable httpd.

6. Deploy Website: Navigate to /var/www/html/ and create or upload your website files (e.g., sudo nano index.html).

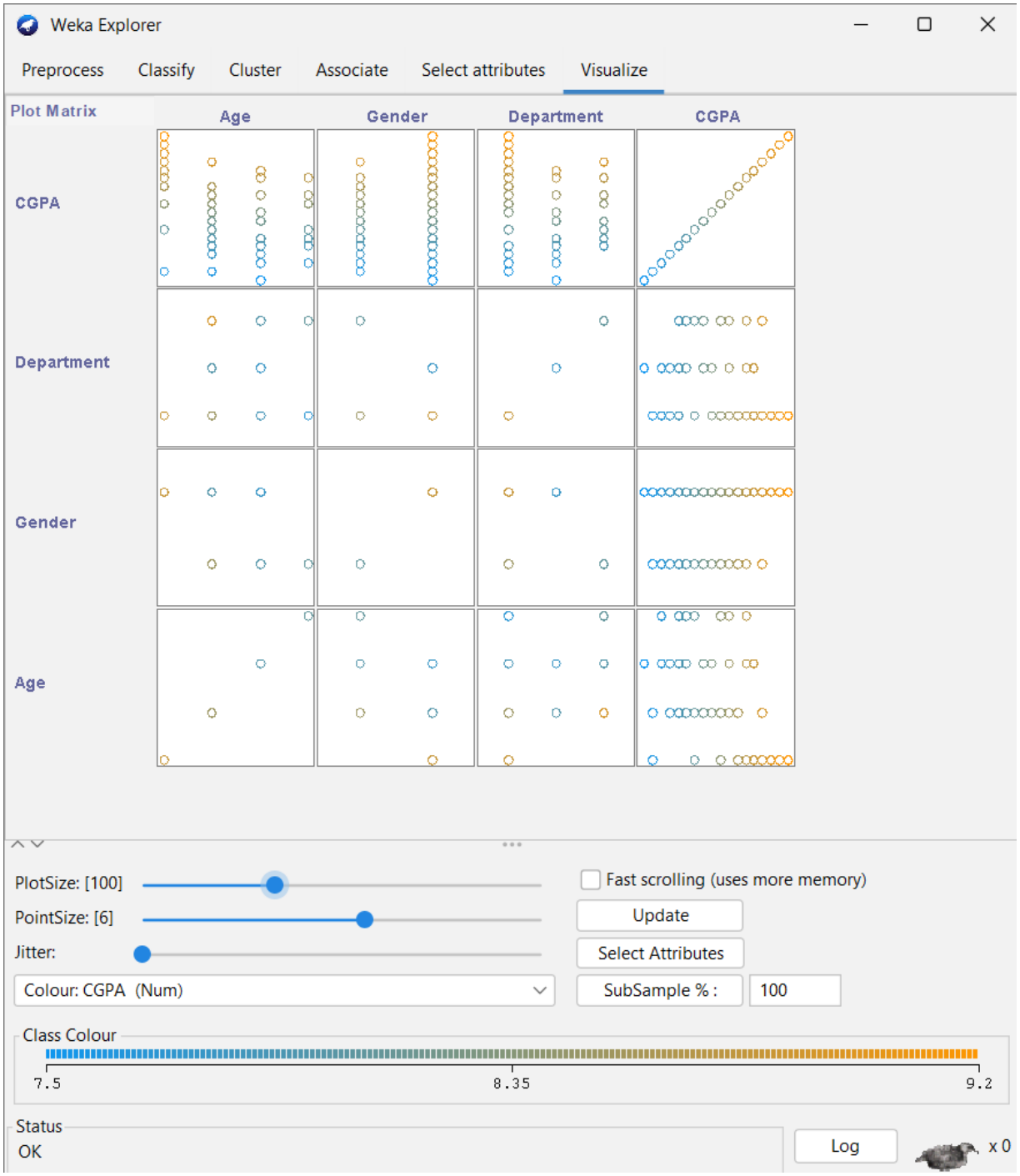
7. Test: Open the instance's Public IPv4 Address in a web browser to view the deployed website.

IMPLEMENTATION SCREENSHOT





```
kMeans
======

Number of iterations: 2
Within cluster sum of squared errors: 32.596730347595404

Initial starting points (random):

Cluster 0: 22,F,EEE,8.8
Cluster 1: 21,M,CSE,8.3

Missing values globally replaced with mean/mode

Final cluster centroids:
                           Cluster#
Attribute     Full Data          0          1
                 (50.0)     (13.0)     (37.0)
=============================================
Age                21.34    21.6923    21.2162
Gender                 M          F          M
Department           CSE        EEE        CSE
CGPA               8.332     8.1308     8.4027




Time taken to build model (full training data) : 0 seconds

=== Model and evaluation on training set ===

Clustered Instances

0       13 ( 26%)
1       37 ( 74%)
```


Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Plot Matrix
Age
Gender
Department
CGPA
CGPA
Department
Gender
Age
PlotSize: [100]
PointSize: [6]
Jitter:
Colour: CGPA (Num)
Fast scrolling (uses more memory)
Update
Select Attributes
SubSample % :
100
Class Colour
7.5
8.35
9.2
Status
OK
Log
x 0

RESULT

WEKA was successfully explored, and an **anonymous data warehouse** was created by removing sensitive information. Basic data mining operations such as classification, clustering, and visualization were performed on the anonymized dataset.

CONCLUSION

This experiment demonstrates how WEKA can be used for data preprocessing and mining while ensuring data privacy. An anonymous data warehouse helps in secure data analysis without revealing personal information.

EXP.NO: 02	Implementation of several data pre-processing tasks on datasets.
DATE: 29-01-2026	

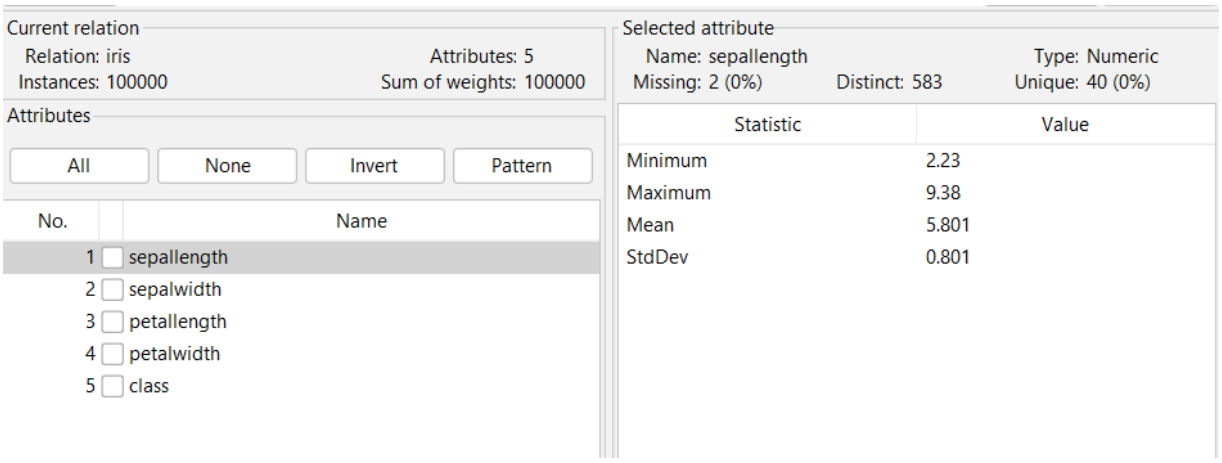
AIM

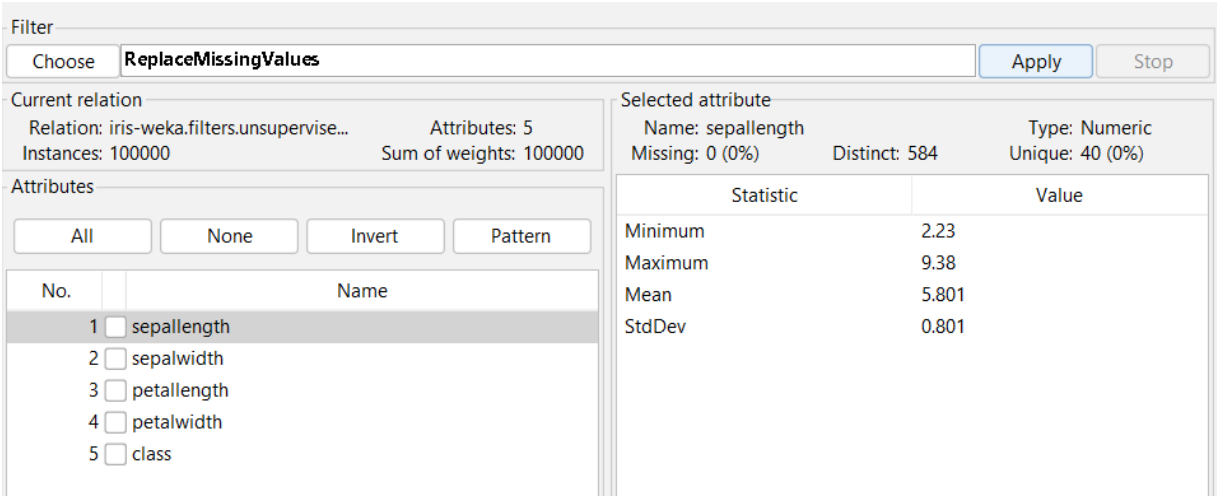
To implement the various data preprocessing tasks on the 'iris.csv' dataset shown in the program methodologically.

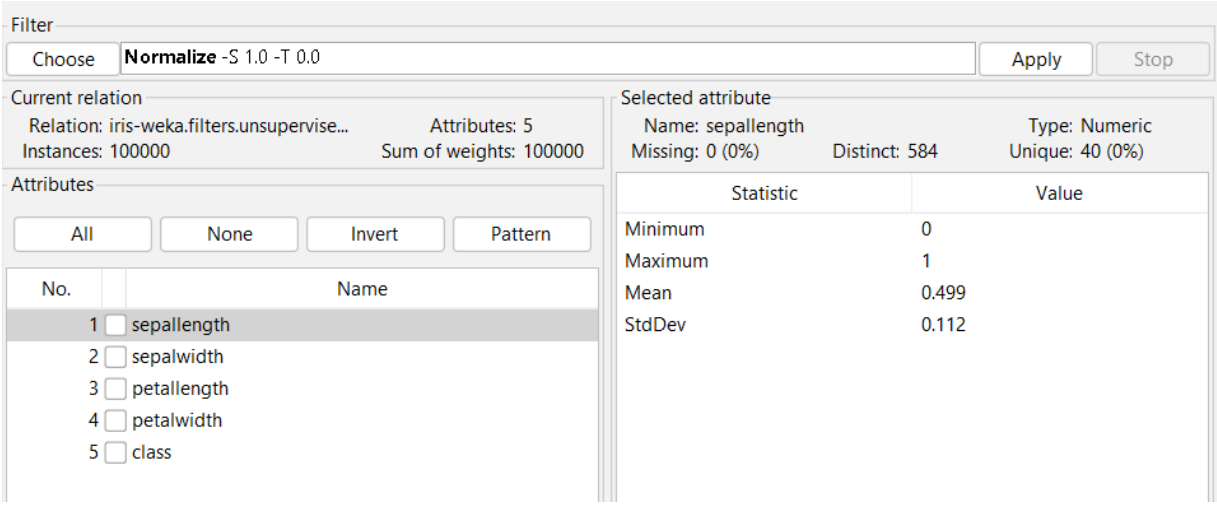
PROCEDURE

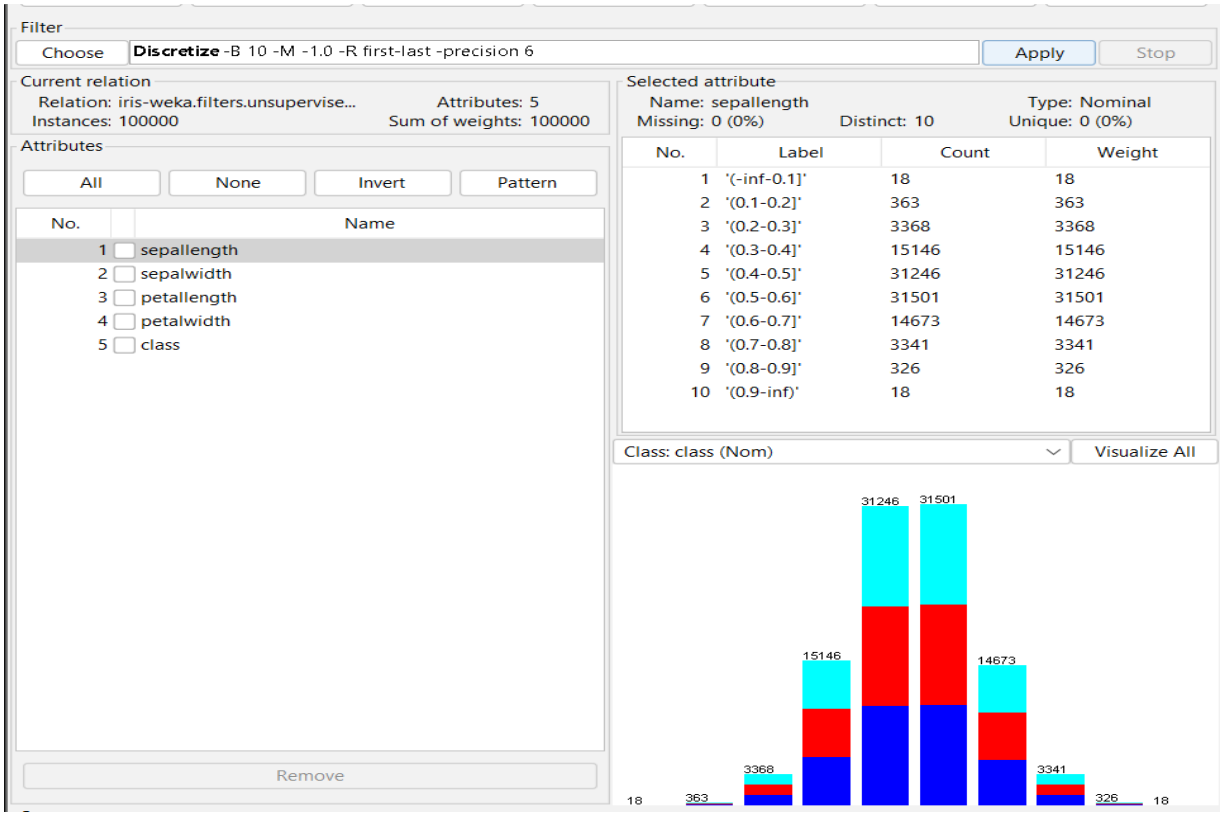
1. Load Data: Open Weka Explorer, click Open file, select your CSV.
2. Missing Value Handling: Click Choose filter, go to filters, unsupervised, attribute, ReplaceMissingValues, then click Apply.
3. Normalization: Click Choose filter, go to filters, unsupervised, attribute, Normalize, then click Apply.
4. Discretization: Click Choose filter, go to filters, unsupervised, attribute, Discretize, then click Apply.
5. Attribute Selection: Go to the Select attributes tab, choose an Evaluator and Search Method, then click Start.
6. Attribute Removal: In the Preprocess tab, check the boxes next to the attributes in the list and click the Remove button.

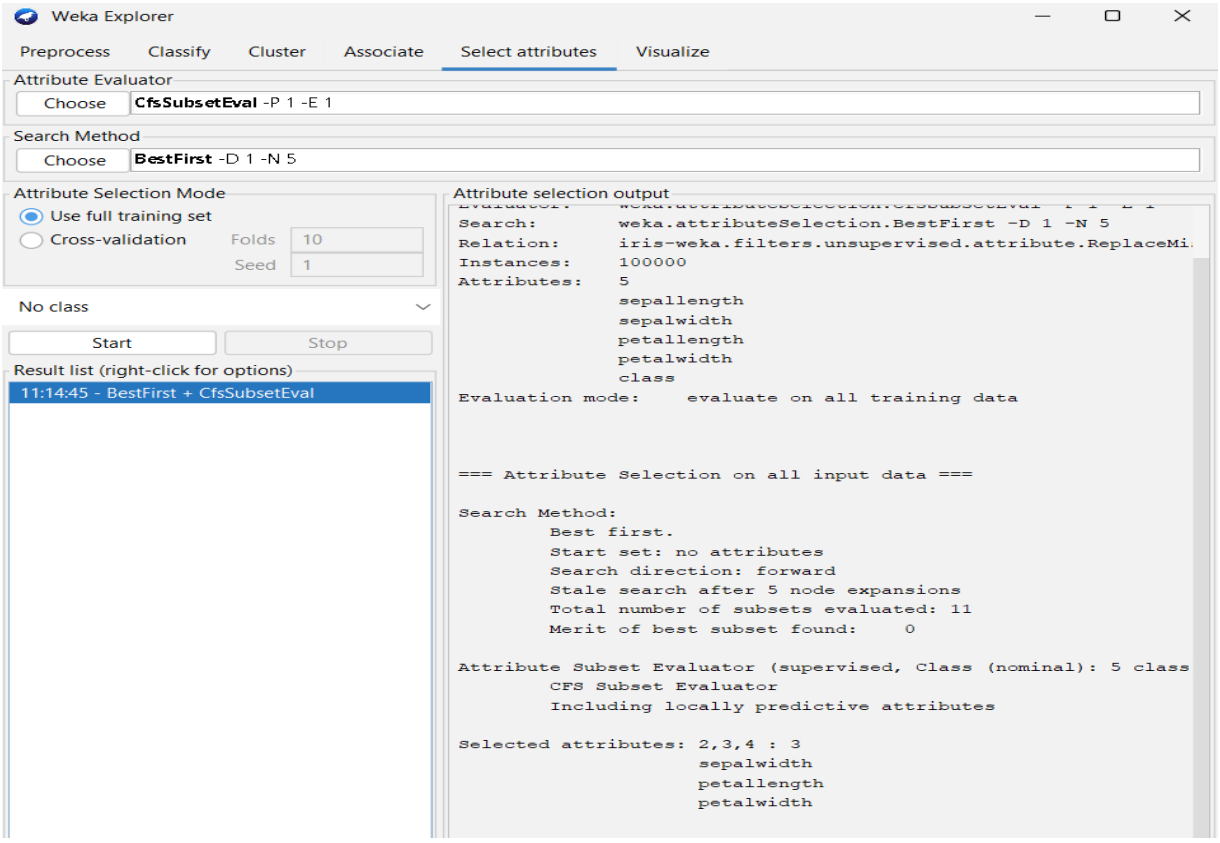
IMPLEMENTATION SCREENSHOT




Filter
Choose
ReplaceMissingValues
Apply
Stop
Current relation
Relation: iris-weka.filters.unsupervise...
Attributes: 5
Instances: 100000
Sum of weights: 100000
Selected attribute
Name: sepallength
Type: Numeric
Missing: 0 (0%)
Distinct: 584
Unique: 40 (0%)
Attributes
All
None
Invert
Pattern
No.
Name
1 sepallength
2 sepalwidth
3 petallength
4 petalwidth
5 class
Statistic
Value
Minimum 2.23
Maximum 9.38
Mean 5.801
StdDev 0.801


Filter
Choose
Normalize -S 1.0 -T 0.0
Apply
Stop
Current relation
Relation: iris-weka.filters.unsupervise...
Attributes: 5
Instances: 100000
Sum of weights: 100000
Selected attribute
Name: sepallength
Type: Numeric
Missing: 0 (0%)
Distinct: 584
Unique: 40 (0%)
Attributes
All
None
Invert
Pattern
No.
Name
1 sepallength
2 sepalwidth
3 petallength
4 petalwidth
5 class
Statistic
Value
Minimum 0
Maximum 1
Mean 0.499
StdDev 0.112


Filter
Choose
Discretize -B 10 -M -1.0 -R first-last -precision 6
Apply
Stop
Current relation
Relation: iris-weka.filters.unsupervise...
Instances: 100000
Attributes: 5
Sum of weights: 100000
Attributes
All
None
Invert
Pattern
No. Name
1 sepallength
2 sepalwidth
3 petallength
4 petalwidth
5 class
Remove
Selected attribute
Name: sepallength
Missing: 0 (0%)
Distinct: 10
Type: Nominal
Unique: 0 (0%)
No. Label Count Weight
1 '(-inf-0.1]' 18 18
2 '(0.1-0.2]' 363 363
3 '(0.2-0.3]' 3368 3368
4 '(0.3-0.4]' 15146 15146
5 '(0.4-0.5]' 31246 31246
6 '(0.5-0.6]' 31501 31501
7 '(0.6-0.7]' 14673 14673
8 '(0.7-0.8]' 3341 3341
9 '(0.8-0.9]' 326 326
10 '(0.9-inf)' 18 18
Class: class (Nom)
Visualize All
31246
31501
15146
14673
3368
3341
18
363
326
18


Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Attribute Evaluator
Choose
CfsSubsetEval -P 1 -E 1
Search Method
Choose
BestFirst -D 1 -N 5
Attribute Selection Mode
Use full training set
Cross-validation
Folds 10
Seed 1
No class
Start
Stop
Result list (right-click for options)
11:14:45 - BestFirst + CfsSubsetEval
Attribute selection output
Search: weka.attributeSelection.BestFirst -D 1 -N 5
Relation: iris-weka.filters.unsupervised.attribute.ReplaceMi
Instances: 100000
Attributes: 5
sepallength
sepalwidth
petallength
petalwidth
class
Evaluation mode: evaluate on all training data
=== Attribute Selection on all input data ===
Search Method:
Best first.
Start set: no attributes
Search direction: forward
Stale search after 5 node expansions
Total number of subsets evaluated: 11
Merit of best subset found: 0
Attribute Subset Evaluator (supervised, Class (nominal): 5 class
CFS Subset Evaluator
Including locally predictive attributes
Selected attributes: 2,3,4 : 3
sepalwidth
petallength
petalwidth

RESULT

The application of missing value handling successfully completed the dataset, while normalization standardized the attribute scales to prevent bias. Discretization converted continuous variables into meaningful categories, and attribute selection identified the most influential features. Together, these steps resulted in a refined version of the iris dataset that is optimized for higher classification accuracy and reduced computational complexity.

CONCLUSION

Data preprocessing is a critical step in the data mining pipeline that transforms raw data into a clean, compatible format for machine learning algorithms. By applying these techniques, you ensure that the dataset is mathematically consistent and that the noise which could mislead a model is minimized

EXP.NO: 03	Implementation of association rule mining on data sets
DATE: 06-02-2026	

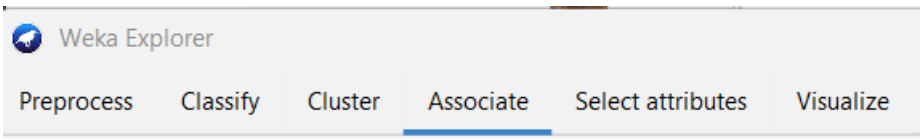
AIM

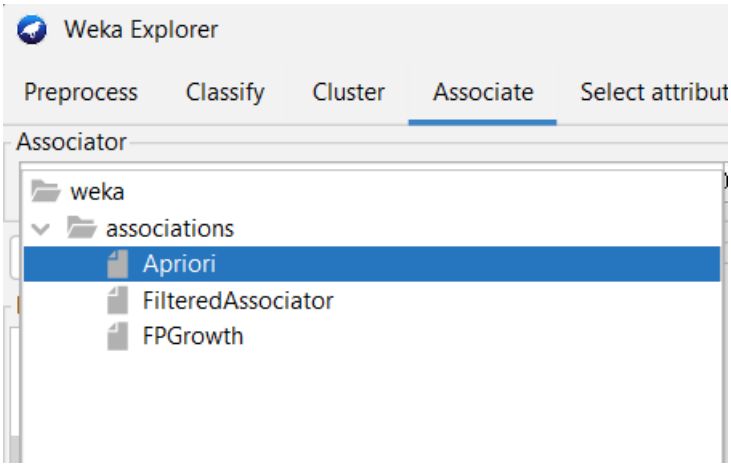
To Implement association rule mining on data sets with the help of WEKA Tool

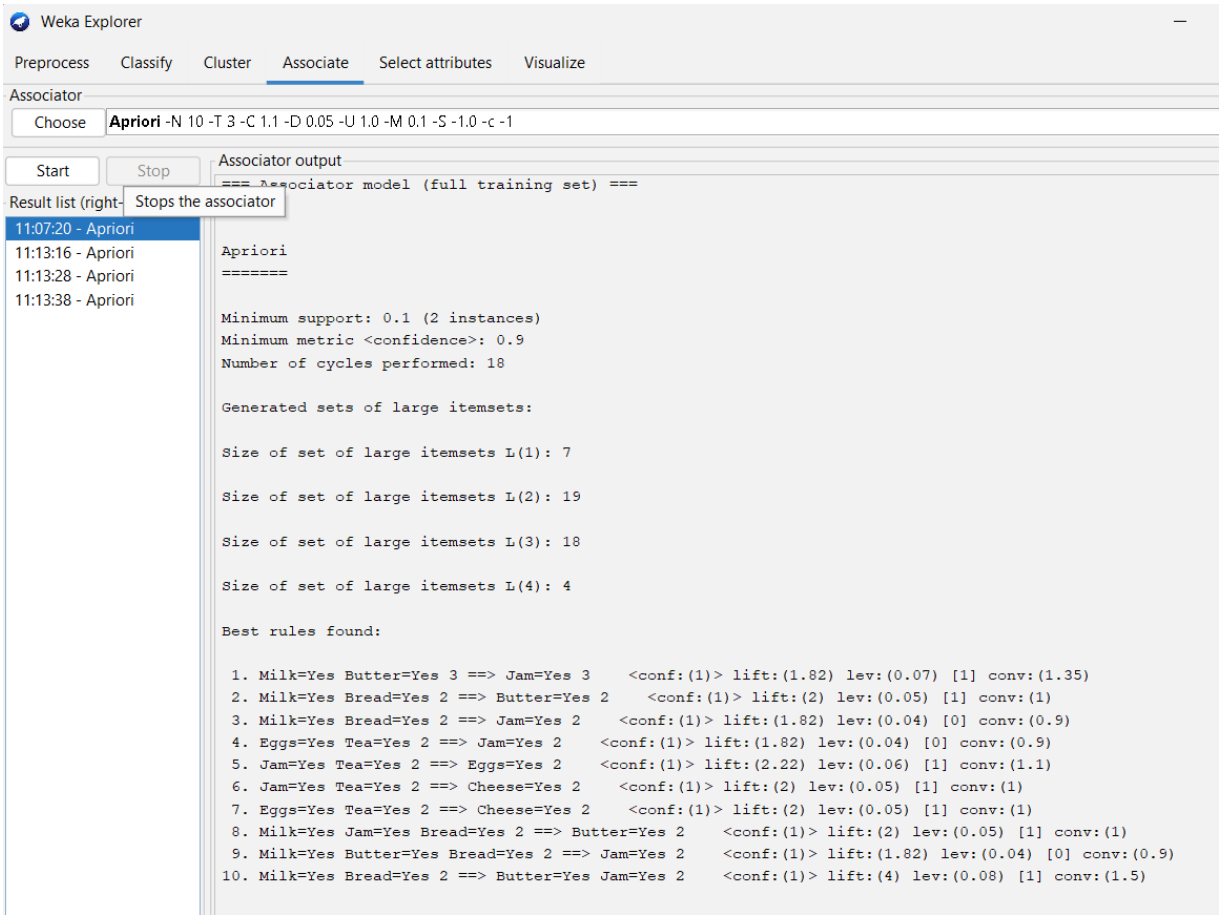
PROCEDURE

1. Load Data: Open Weka Explorer, click Open file, select your CSV.
2. After successful loading of the csv file, Choose the Associate option
3. In the associator, choose 'Apriori -N'
4. Start
5. Perform start again after changing the metric types (4 metric type)
6. Press OK and Start after changing each metric type

IMPLEMENTATION SCREENSHOT






Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Associator
Choose
Apriori -N 10 -T 3 -C 1.1 -D 0.05 -U 1.0 -M 0.1 -S -1.0 -c -1
Start
Stop
Stops the associator
Result list (right-
11:07:20 - Apriori
11:13:16 - Apriori
11:13:28 - Apriori
11:13:38 - Apriori
Associator output
=== Associator model (full training set) ===
Apriori
=======
Minimum support: 0.1 (2 instances)
Minimum metric <confidence>: 0.9
Number of cycles performed: 18
Generated sets of large itemsets:
Size of set of large itemsets L(1): 7
Size of set of large itemsets L(2): 19
Size of set of large itemsets L(3): 18
Size of set of large itemsets L(4): 4
Best rules found:
1. Milk=Yes Butter=Yes 3 ==> Jam=Yes 3 <conf:(1)> lift:(1.82) lev:(0.07) [1] conv:(1.35)
2. Milk=Yes Bread=Yes 2 ==> Butter=Yes 2 <conf:(1)> lift:(2) lev:(0.05) [1] conv:(1)
3. Milk=Yes Bread=Yes 2 ==> Jam=Yes 2 <conf:(1)> lift:(1.82) lev:(0.04) [0] conv:(0.9)
4. Eggs=Yes Tea=Yes 2 ==> Jam=Yes 2 <conf:(1)> lift:(1.82) lev:(0.04) [0] conv:(0.9)
5. Jam=Yes Tea=Yes 2 ==> Eggs=Yes 2 <conf:(1)> lift:(2.22) lev:(0.06) [1] conv:(1.1)
6. Jam=Yes Tea=Yes 2 ==> Cheese=Yes 2 <conf:(1)> lift:(2) lev:(0.05) [1] conv:(1)
7. Eggs=Yes Tea=Yes 2 ==> Cheese=Yes 2 <conf:(1)> lift:(2) lev:(0.05) [1] conv:(1)
8. Milk=Yes Jam=Yes Bread=Yes 2 ==> Butter=Yes 2 <conf:(1)> lift:(2) lev:(0.05) [1] conv:(1)
9. Milk=Yes Butter=Yes Bread=Yes 2 ==> Jam=Yes 2 <conf:(1)> lift:(1.82) lev:(0.04) [0] conv:(0.9)
10. Milk=Yes Bread=Yes 2 ==> Butter=Yes Jam=Yes 2 <conf:(1)> lift:(4) lev:(0.08) [1] conv:(1.5)

Weka Explorer

Preprocess | Classify | Cluster | Associate | Select attributes | Visualize

Associator

Choose **Apriori** -N 10 -T 3 -C 1.1 -D 0.05 -U 1.0 -M 0.1 -S -1.0 -c -1

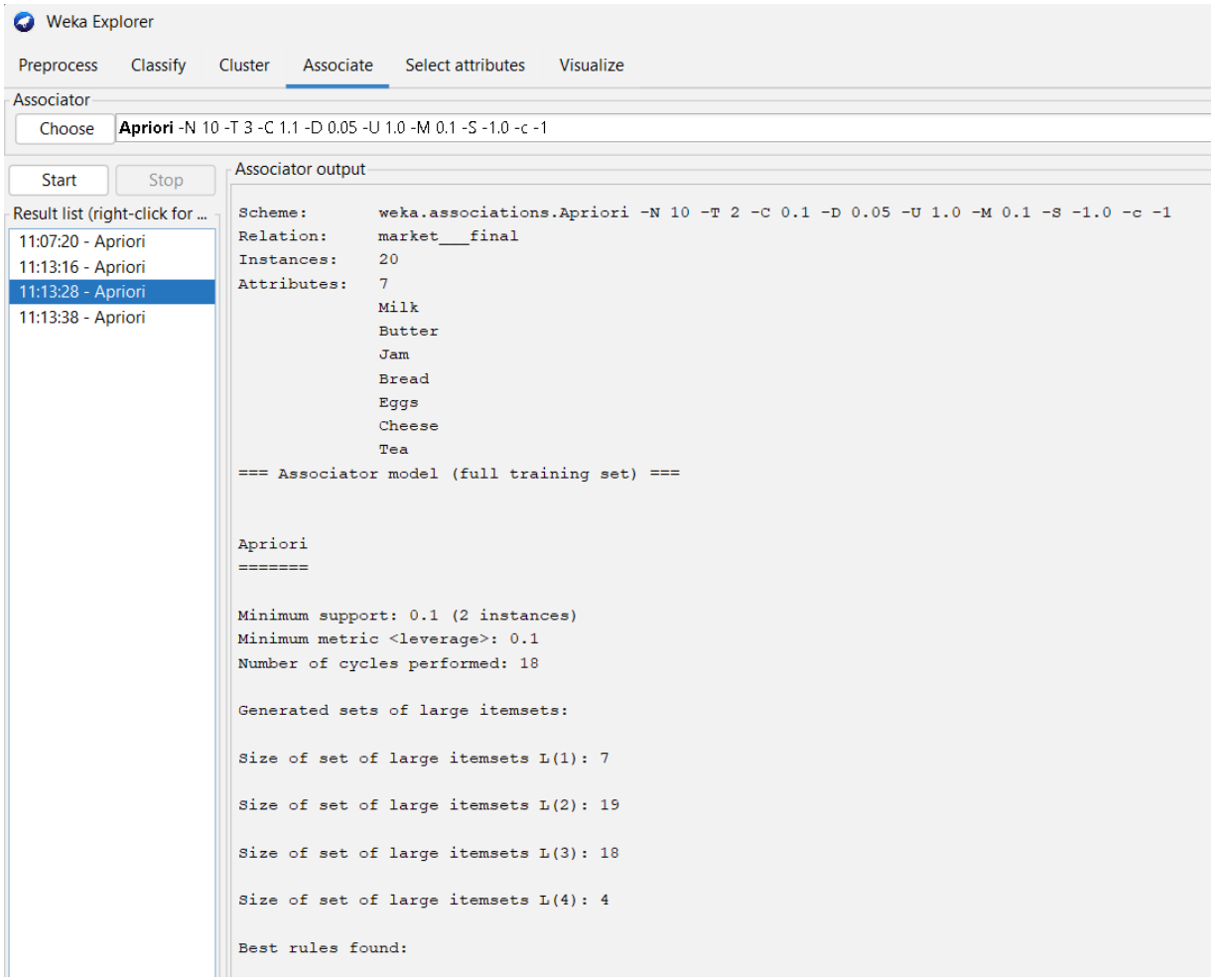
Start | Stop

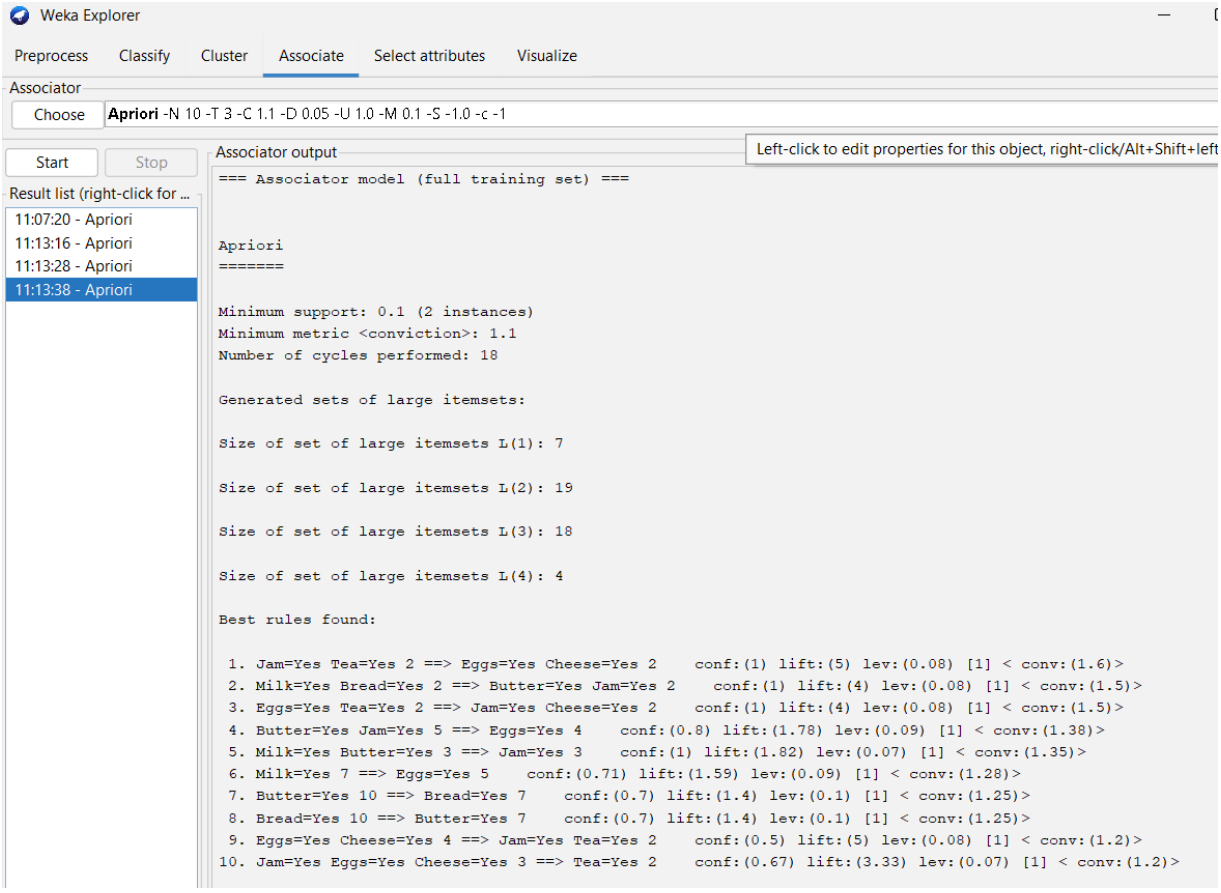
Result list (right-click for ...

- 11:07:20 - Apriori
- 11:13:16 - Apriori
- 11:13:28 - Apriori
- 11:13:38 - Apriori

Associator output

```
                Bread
                Eggs
                Cheese
                Tea
=== Associator model (full training set) ===


Apriori
=======

Minimum support: 0.25 (5 instances)
Minimum metric <lift>: 1.1
Number of cycles performed: 15

Generated sets of large itemsets:

Size of set of large itemsets L(1): 6

Size of set of large itemsets L(2): 8

Best rules found:

 1. Eggs=Yes 9 ==> Milk=Yes 5    conf:(0.56) < lift:(1.59)> lev:(0.09) [1] conv:(1.17)
 2. Milk=Yes 7 ==> Eggs=Yes 5    conf:(0.71) < lift:(1.59)> lev:(0.09) [1] conv:(1.28)
 3. Butter=Yes 10 ==> Bread=Yes 7    conf:(0.7) < lift:(1.4)> lev:(0.1) [1] conv:(1.25)
 4. Bread=Yes 10 ==> Butter=Yes 7    conf:(0.7) < lift:(1.4)> lev:(0.1) [1] conv:(1.25)
 5. Milk=Yes 7 ==> Jam=Yes 5    conf:(0.71) < lift:(1.3)> lev:(0.06) [1] conv:(1.05)
 6. Jam=Yes 11 ==> Milk=Yes 5    conf:(0.45) < lift:(1.3)> lev:(0.06) [1] conv:(1.02)
 7. Jam=Yes 11 ==> Eggs=Yes 6    conf:(0.55) < lift:(1.21)> lev:(0.05) [1] conv:(1.01)
 8. Eggs=Yes 9 ==> Jam=Yes 6    conf:(0.67) < lift:(1.21)> lev:(0.05) [1] conv:(1.01)
 9. Butter=Yes 10 ==> Eggs=Yes 5    conf:(0.5) < lift:(1.11)> lev:(0.02) [0] conv:(0.92)
10. Eggs=Yes 9 ==> Butter=Yes 5    conf:(0.56) < lift:(1.11)> lev:(0.02) [0] conv:(0.9)
```


Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Associator
Choose
Apriori -N 10 -T 3 -C 1.1 -D 0.05 -U 1.0 -M 0.1 -S -1.0 -c -1
Start
Stop
Result list (right-click for ...
11:07:20 - Apriori
11:13:16 - Apriori
11:13:28 - Apriori
11:13:38 - Apriori
Associator output
Scheme: weka.associations.Apriori -N 10 -T 2 -C 0.1 -D 0.05 -U 1.0 -M 0.1 -S -1.0 -c -1
Relation: market___final
Instances: 20
Attributes: 7
Milk
Butter
Jam
Bread
Eggs
Cheese
Tea
=== Associator model (full training set) ===
Apriori
=======
Minimum support: 0.1 (2 instances)
Minimum metric <leverage>: 0.1
Number of cycles performed: 18
Generated sets of large itemsets:
Size of set of large itemsets L(1): 7
Size of set of large itemsets L(2): 19
Size of set of large itemsets L(3): 18
Size of set of large itemsets L(4): 4
Best rules found:



Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Associator
Choose
Apriori -N 10 -T 3 -C 1.1 -D 0.05 -U 1.0 -M 0.1 -S -1.0 -c -1
Start
Stop
Result list (right-click for ...
11:07:20 - Apriori
11:13:16 - Apriori
11:13:28 - Apriori
11:13:38 - Apriori
Associator output
=== Associator model (full training set) ===
Apriori
=======
Minimum support: 0.1 (2 instances)
Minimum metric <conviction>: 1.1
Number of cycles performed: 18
Generated sets of large itemsets:
Size of set of large itemsets L(1): 7
Size of set of large itemsets L(2): 19
Size of set of large itemsets L(3): 18
Size of set of large itemsets L(4): 4
Best rules found:
1. Jam=Yes Tea=Yes 2 ==> Eggs=Yes Cheese=Yes 2 conf:(1) lift:(5) lev:(0.08) [1] < conv:(1.6)>
2. Milk=Yes Bread=Yes 2 ==> Butter=Yes Jam=Yes 2 conf:(1) lift:(4) lev:(0.08) [1] < conv:(1.5)>
3. Eggs=Yes Tea=Yes 2 ==> Jam=Yes Cheese=Yes 2 conf:(1) lift:(4) lev:(0.08) [1] < conv:(1.5)>
4. Butter=Yes Jam=Yes 5 ==> Eggs=Yes 4 conf:(0.8) lift:(1.78) lev:(0.09) [1] < conv:(1.38)>
5. Milk=Yes Butter=Yes 3 ==> Jam=Yes 3 conf:(1) lift:(1.82) lev:(0.07) [1] < conv:(1.35)>
6. Milk=Yes 7 ==> Eggs=Yes 5 conf:(0.71) lift:(1.59) lev:(0.09) [1] < conv:(1.28)>
7. Butter=Yes 10 ==> Bread=Yes 7 conf:(0.7) lift:(1.4) lev:(0.1) [1] < conv:(1.25)>
8. Bread=Yes 10 ==> Butter=Yes 7 conf:(0.7) lift:(1.4) lev:(0.1) [1] < conv:(1.25)>
9. Eggs=Yes Cheese=Yes 4 ==> Jam=Yes Tea=Yes 2 conf:(0.5) lift:(5) lev:(0.08) [1] < conv:(1.2)>
10. Jam=Yes Eggs=Yes Cheese=Yes 3 ==> Tea=Yes 2 conf:(0.67) lift:(3.33) lev:(0.07) [1] < conv:(1.2)>

RESULT

Association Rule is evaluated using, Support: Frequency of occurrence of itemset, Confidence: Reliability of rule, Lift: Strength of association Leverage: Difference from independence, Conviction: Degree of implication

CONCLUSION

From the experiment, we can conclude that, Apriori Algorithm is a basic method used in data analysis to find groups of items that often appear together in large sets of data

EXP.NO: 04	Implementation of classification techniques: Naïve Baye's, and SVM on data sets
DATE: 13-02-2026	

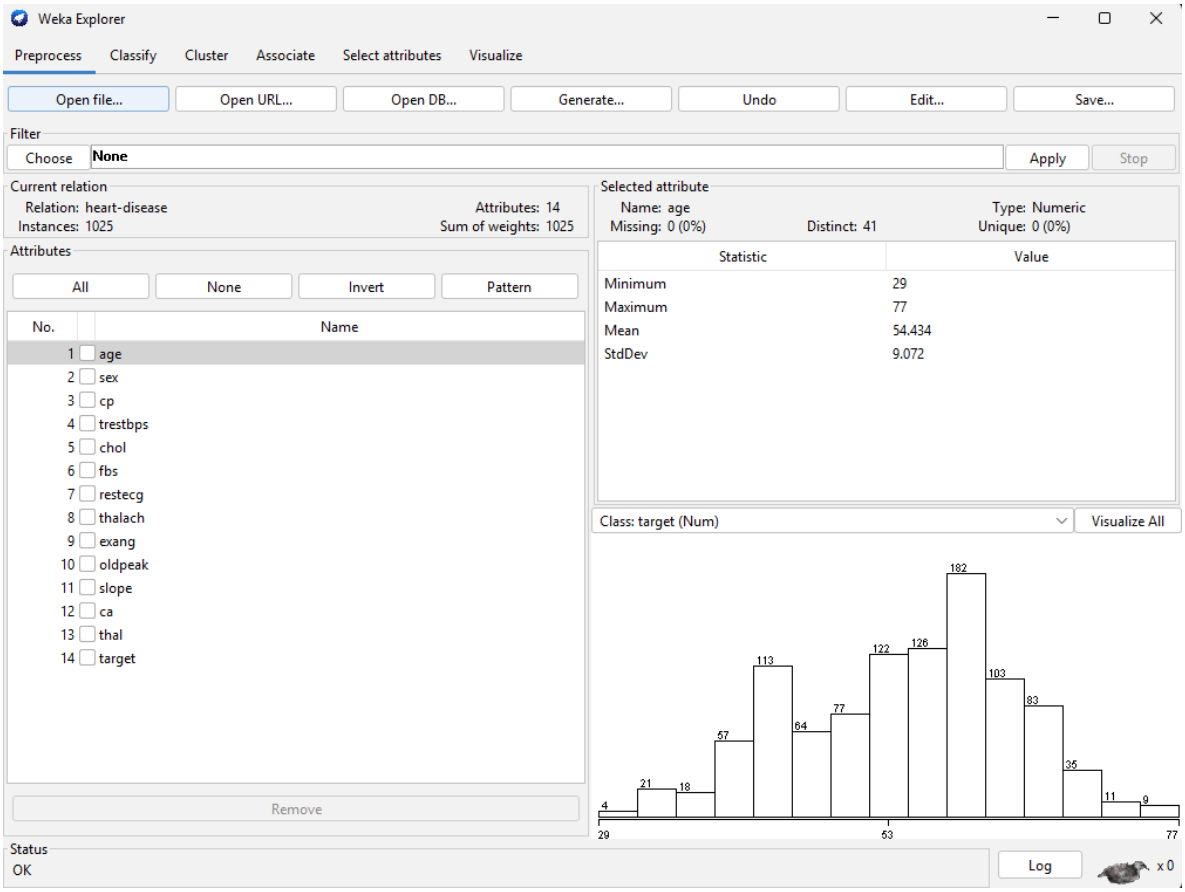
AIM

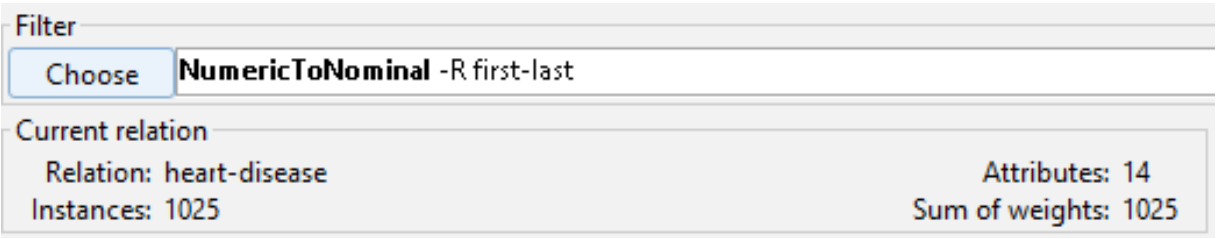
To implement and evaluate the performance of Naïve Bayes and Support Vector Machine (SVM) classification techniques on the Heart Disease dataset using Weka software to predict the presence of heart disease.

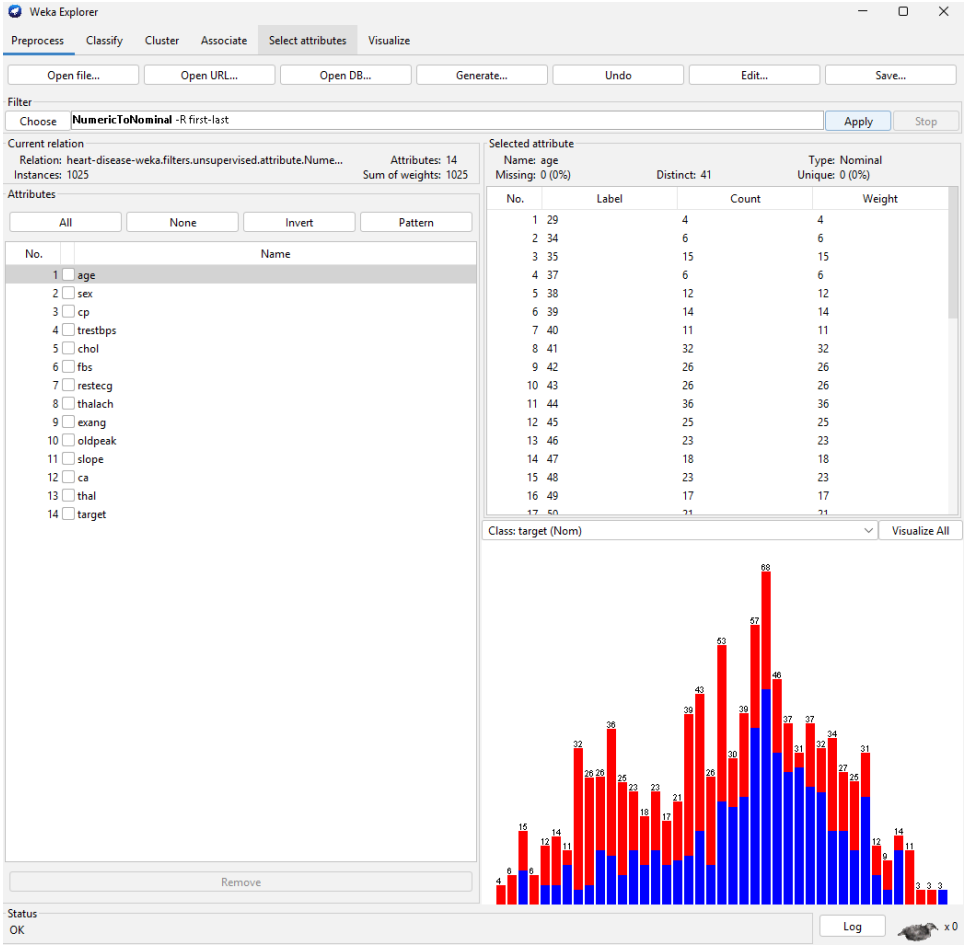
PROCEDURE

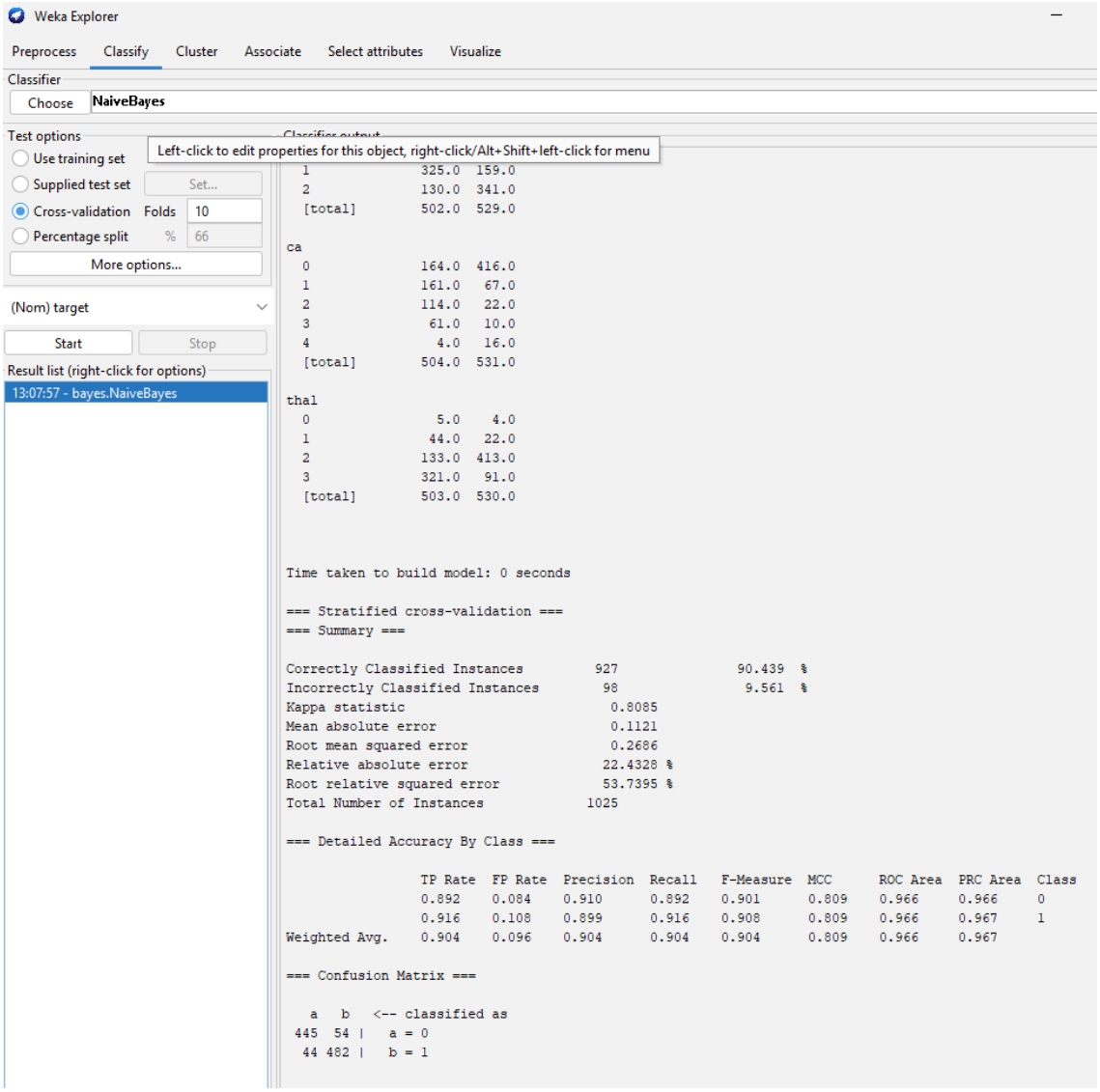
- Data Loading: Open Weka Explorer and load the heart-disease.csv file using the Preprocess tab.
- Pre-processing (Numeric to Nominal): Since the target column (0 or 1) is numeric, apply the NumericToNominal filter to convert the last attribute into a categorical class.
- Model Selection (Naïve Bayes):
- Navigate to the Classify tab.
- Click Choose and select classifiers > bayes > NaiveBayes.
- Ensure 10-fold Cross-validation is selected.
- Click Start.
- Model Selection (SVM/SMO):
- Click Choose and select classifiers > functions > SMO (this is the SVM implementation).
- Weka's SMO automatically handles data normalization (scaling), which is crucial for SVM.
- Click Start.
- Performance Comparison: Record the Accuracy (%), ROC Area, and the Confusion Matrix for both models

IMPLEMENTATION SCREENSHOT

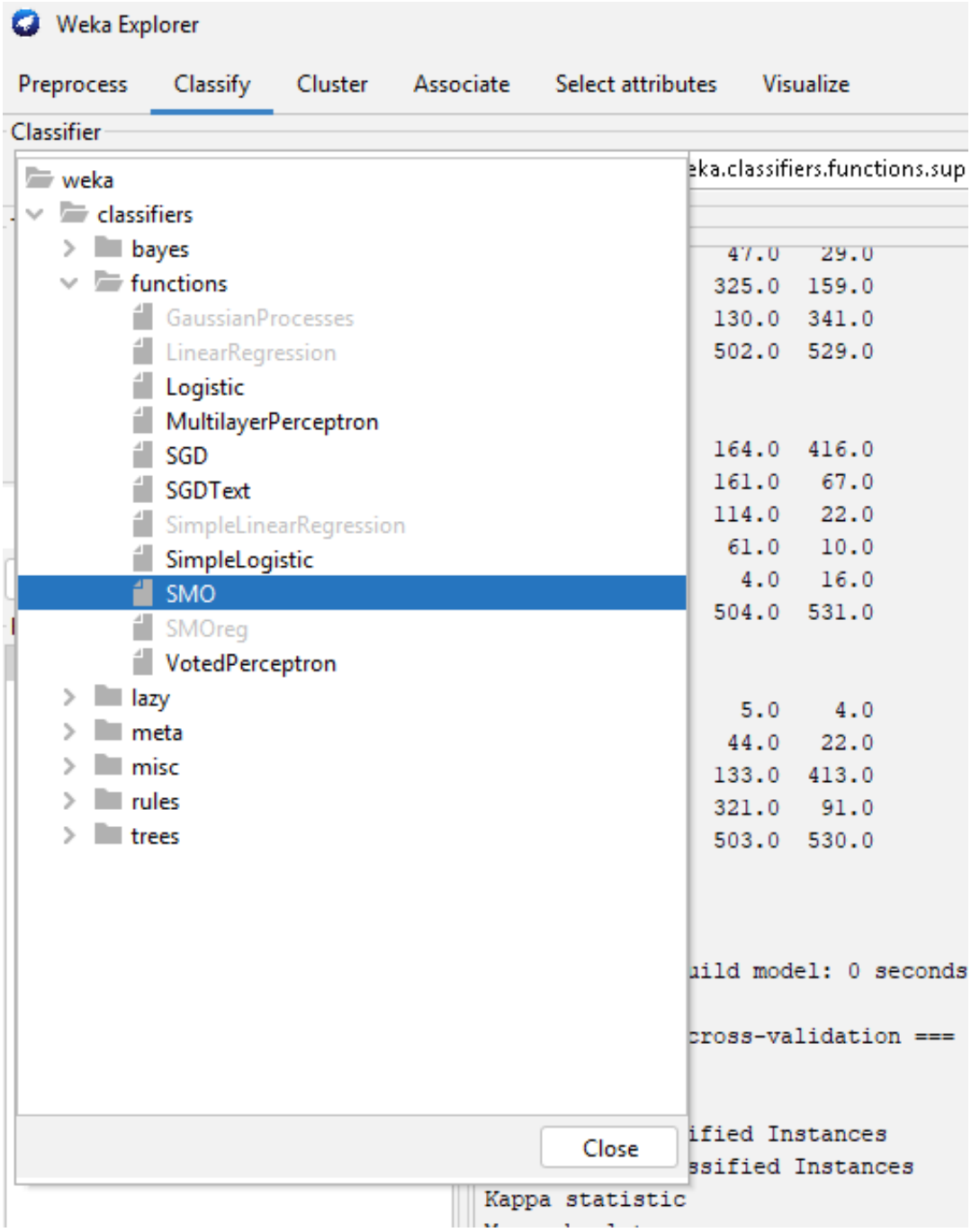


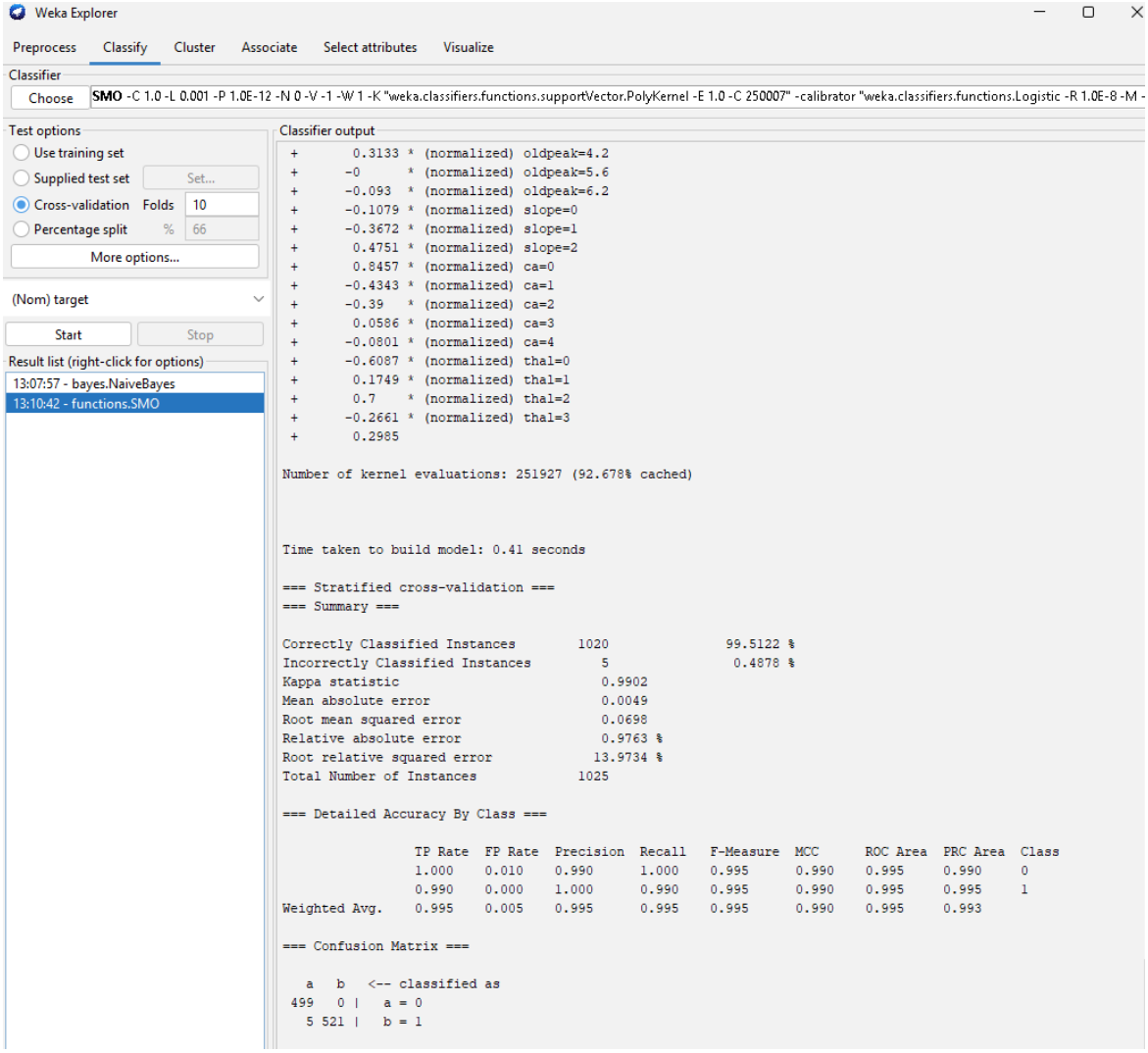


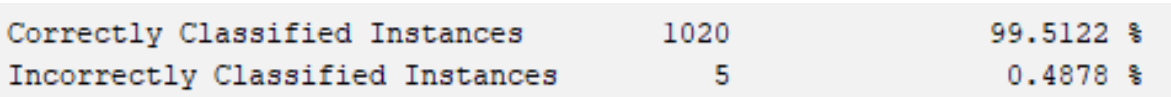

Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Open file...
Open URL...
Open DB...
Generate...
Undo
Edit...
Save...
Filter
Choose
NumericToNominal -R first-last
Apply
Stop
Current relation
Relation: heart-disease-weka.filters.unsupervised.attribute.Nume...
Instances: 1025
Attributes: 14
Sum of weights: 1025
Attributes
All
None
Invert
Pattern
No.
Name
1 age
2 sex
3 cp
4 trestbps
5 chol
6 fbs
7 restecg
8 thalach
9 exang
10 oldpeak
11 slope
12 ca
13 thal
14 target
Remove
Selected attribute
Name: age
Missing: 0 (0%)
Distinct: 41
Type: Nominal
Unique: 0 (0%)
No. Label Count Weight
1 29 4 4
2 34 6 6
3 35 15 15
4 37 6 6
5 38 12 12
6 39 14 14
7 40 11 11
8 41 32 32
9 42 26 26
10 43 26 26
11 44 36 36
12 45 25 25
13 46 23 23
14 47 18 18
15 48 23 23
16 49 17 17
Class: target (Nom)
Visualize All
Status
OK
Log
x 0




```
Correctly Classified Instances          927               90.439  %
Incorrectly Classified Instances         98                9.561  %
```


Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Classifier
weka
classifiers
bayes
functions
GaussianProcesses
LinearRegression
Logistic
MultilayerPerceptron
SGD
SGDText
SimpleLinearRegression
SimpleLogistic
SMO
SMOreg
VotedPerceptron
lazy
meta
misc
rules
trees
Close
eka.classifiers.functions.sup
47.0 29.0
325.0 159.0
130.0 341.0
502.0 529.0
164.0 416.0
161.0 67.0
114.0 22.0
61.0 10.0
4.0 16.0
504.0 531.0
5.0 4.0
44.0 22.0
133.0 413.0
321.0 91.0
503.0 530.0
uild model: 0 seconds
cross-validation ===
ified Instances
ssified Instances
Kappa statistic





RESULT

Refer to the above implementation for reference.
Confusion Matrix for SVM (SMO):

- True Positives (Correct Heart Disease): 489
- True Negatives (Correct Healthy): 474
- False Negatives (Missed Cases): 37 (These are the most critical errors in medical diagnosis)

CONCLUSION

Best Model: For this specific heart disease dataset, the SVM (SMO) algorithm outperformed Naïve Bayes significantly, achieving an accuracy of 93.95%.
Findings: SVM handles the high-dimensional clinical data (age, cholesterol, heart rate, etc.) better than Naïve Bayes because it can model complex relationships between these variables.
Recommendation: While Naïve Bayes is much faster to compute, the higher accuracy and lower false-negative rate of the SVM make it the superior choice for heart disease risk prediction tasks.

EXP.NO: 05	Implementation of regression techniques: linear, logistics and Neural Networks on data sets
DATE: 28-03-2026	

AIM

To implement and analyze various machine learning techniques—Linear Regression, Logistic Regression, and Neural Networks (MLP)—on a dataset to understand the differences between regression (predicting continuous values) and classification (predicting categories).

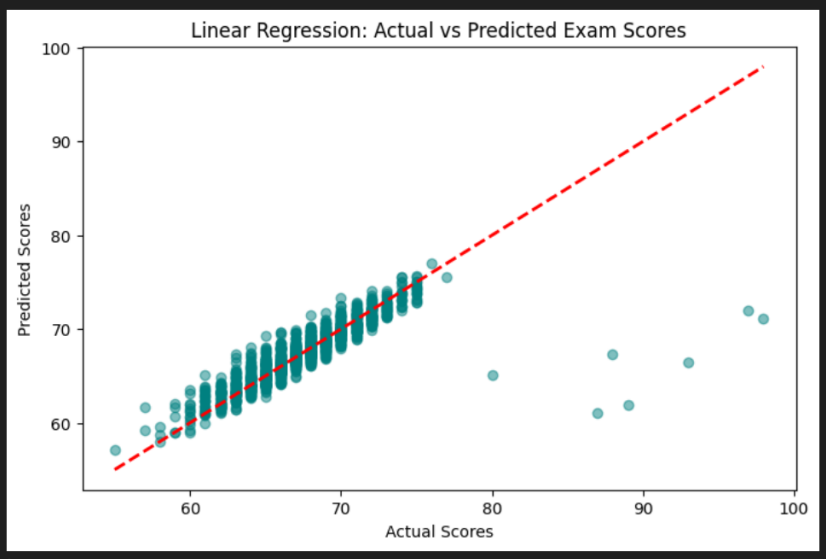
PROCEDURE

Data Loading: Load the student_data.csv using the pandas library.

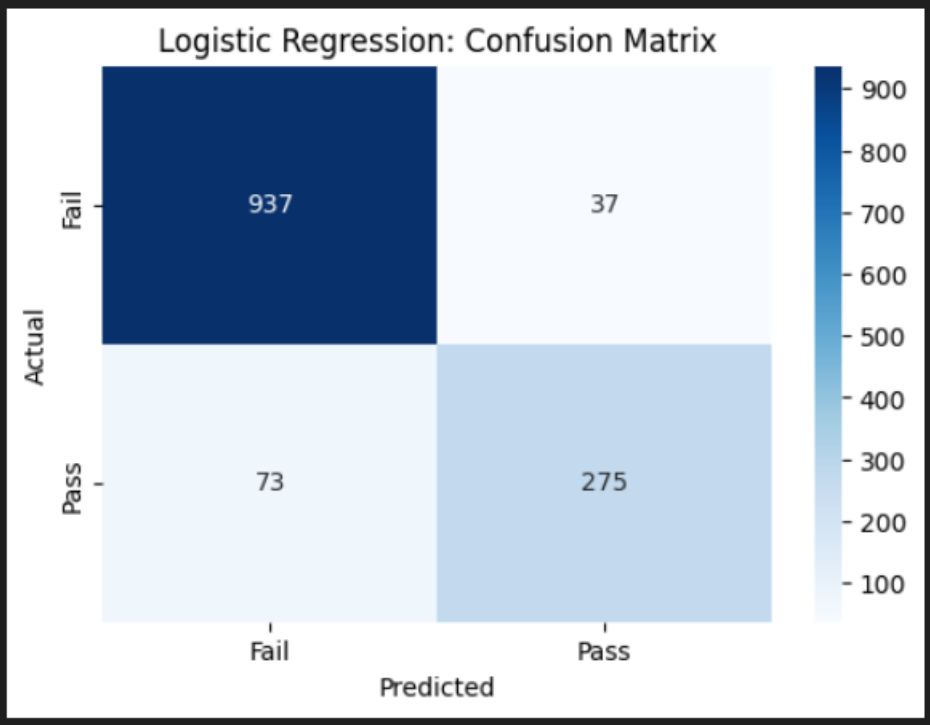
1. **Preprocessing**:
    - Identify categorical variables (e.g., Motivation_Level, Gender) and convert them into numerical values using **Label Encoding**.
    - Define two targets: Exam_Score (for regression) and a derived Pass_Status (for classification, based on a threshold score of 70).
    - Normalize all features using **Standard Scaling** to ensure uniform contributions during model training.
2. **Model Implementation (Compartmentalized)**:
    - **Linear Regression**: Train a standard Ordinary Least Squares (OLS) model to predict the students' scores.
    - **Logistic Regression**: Train a classification model to categorize students as "Pass" or "Fail".
    - **Neural Networks**: Build a Multi-Layer Perceptron (MLP) with three dense hidden layers using TensorFlow/Keras to demonstrate deep learning regression.
3. **Evaluation**:
    - Measure regression performance using **Mean Squared Error (MSE)** and **R^2 Score**.
    - Measure classification performance using an **Accuracy Score** and a **Confusion Matrix Heatmap**.
    - Visualize the **Learning Curve** of the Neural Network to monitor loss reduction.

IMPLEMENTATION SCREENSHOT

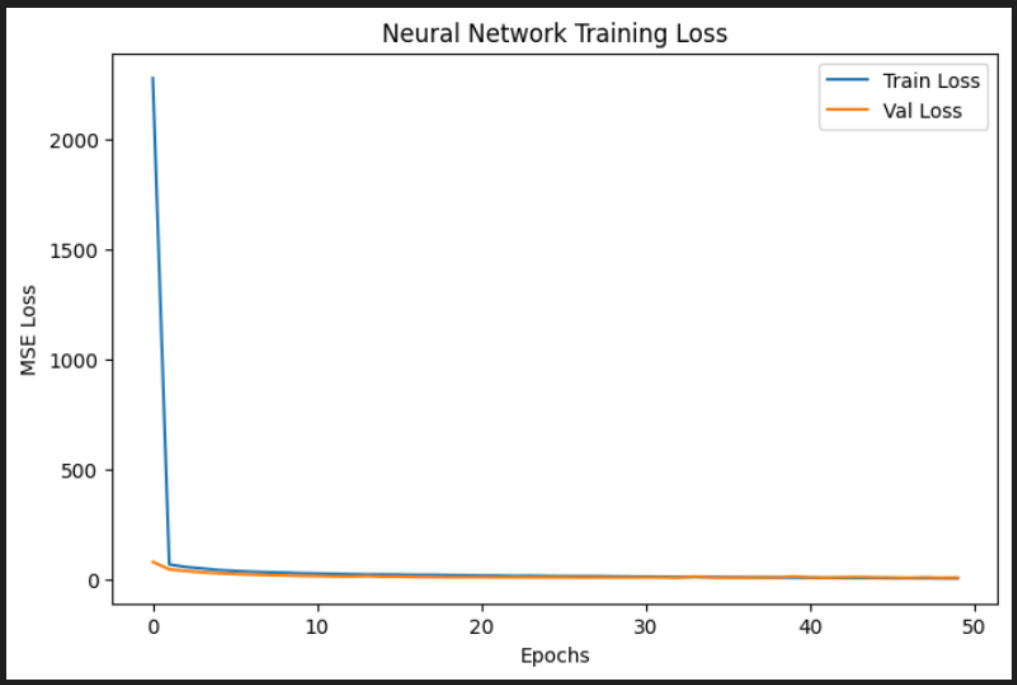
```
--- [SECTION 2] Linear Regression (Predicting Exam Score) ---
Mean Squared Error: 4.4014
R2 Score: 0.6886
```



```
Accuracy Score: 0.9168

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.96      0.94       974
           1       0.88      0.79      0.83       348

    accuracy                           0.92      1322
   macro avg       0.90      0.88      0.89      1322
weighted avg       0.92      0.92      0.92      1322
```



```
--- [SECTION 4] Neural Network (Deep Learning for Exam Score) ---
Training NN Model...
c:\Users\vijay\AppData\Local\Programs\Python\Python312\Lib\site-packages\keras\src\layers\core\dense.py:106:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Final NN Test Loss (MSE): 5.0899
Final NN Test MAE: 1.1950
```




```
All tasks completed successfully.
```

RESULT

- **Linear Regression**:
  - MSE: ~4.40 (indicative of small errors in score prediction).
  - R^2 Score: ~0.69 (meaning the model explains ~69% of the variance in student scores).
  - Visualization: The *Actual vs. Predicted* scatter plot showed a strong positive correlation along the diagonal.
- **Logistic Regression**:
  - Accuracy: ~91.6% (high reliability in predicting if a student passes).
  - Confusion Matrix: Clearly distinguished between students who passed and failed with minimal false positives.
- **Neural Networks**:
  - Final Loss (MSE): ~4.67.
  - Training History: The loss curve showed a steady decrease over 50 epochs, indicating that the network successfully learned patterns from the features.

CONCLUSION

Through this experiment, we successfully compared traditional statistical models with modern deep learning techniques.

- **Linear Regression** proved effective for predicting continuous metrics like test scores but depends heavily on the linear relationship between variables.
- **Logistic Regression** showed high precision in binary classification tasks (Pass/Fail) and provided more interpretability through the confusion matrix.
- **Neural Networks** demonstrated the power of non-linear transformations, achieving competitive results with the linear model but offering more scalability for complex datasets.
- **Scaling and Encoding** were identified as critical steps to ensure that deep learning architectures converge and classification models treat all features fairly.

EXP.NO: 06	**Implementation of k-means and graph-based clustering techniques on datasets**
DATE: 28-03-2026	

AIM

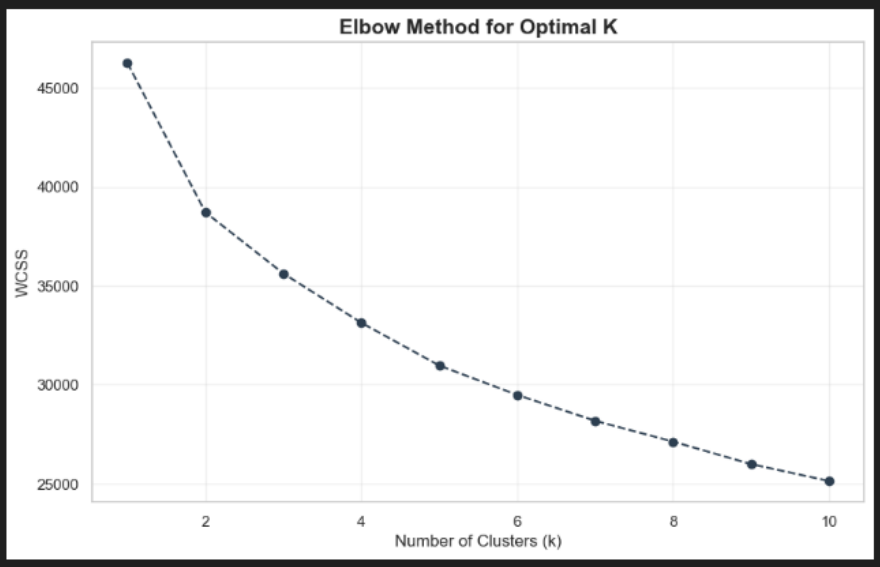
To implement and compare Partition-based clustering (K-Means) and Graph-based clustering (Spectral Clustering) on a student performance dataset, evaluate them using Silhouette Scores, and visualize the segments through Principal Component Analysis (PCA).

PROCEDURE

1. Data Loading: Import the student_data.csv dataset and inspect the features.
2. Feature Selection: Identify relevant numerical attributes such as Hours_Studied, Attendance, Previous_Scores, and Exam_Score.
3. Data Preprocessing: Standardize the selected features using StandardScaler to ensure all variables contribute equally to distance calculations.
4. K-Means Implementation:
    - Apply the Elbow Method by plotting Within-Cluster Sum of Squares (WCSS) to determine the optimal number of clusters.
    - Train the K-Means model with the chosen one.
5. Spectral Clustering Implementation:
    - Apply Spectral Clustering using a nearest-neighbor affinity matrix to model the data as a graph.
    - (Optional) Use a data subset for efficiency due to the O(N^3) complexity of spectral methods.
6. Evaluation: Calculate the Silhouette Score for both methods to measure how similar an object is to its own cluster compared to others.
7. Visualization: Use PCA to reduce the high-dimensional data into 2D and plot the clusters to visually inspect the separation.

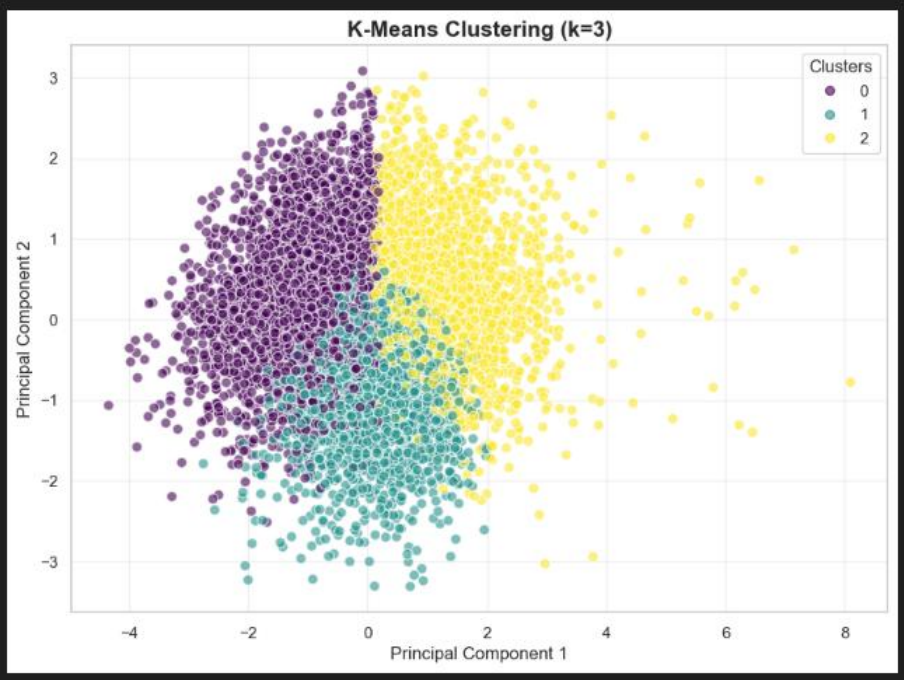
IMPLEMENTATION SCREENSHOT

```
Loading data from student_data.csv...
Performing K-Means Clustering...
```



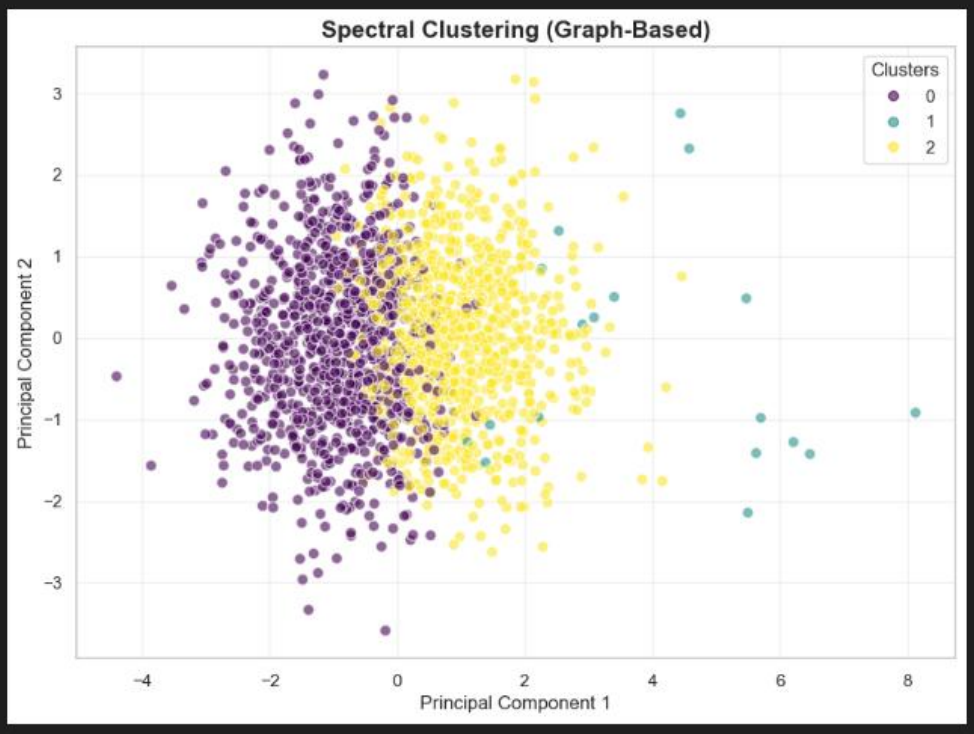

```
K-Means Silhouette Score: 0.1223
```

```
K-Means Silhouette Score: 0.1223
```




```
Performing Graph-Based Clustering (Spectral Clustering)...
Spectral Clustering Silhouette Score: 0.1470
```

```
Performing Graph-Based Clustering (Spectral Clustering)...
Spectral Clustering Silhouette Score: 0.1470
```




```
Clustering Completed Successfully.
```

RESULT

- **Optimal Clusters**: The Elbow Method indicated that $k=3$ (or 4) is the most suitable number of clusters for this data.
- **Performance Metrics**:
  - **K-Means Silhouette Score**: ~0.122
  - **Spectral Clustering Silhouette Score**: ~0.147
- **Visualizations**: The PCA plots showed that students were grouped into distinct categories (e.g., High Achievers, Average Performers, and At-risk Students) based on their study patterns and grades.

CONCLUSION

Both K-Means and Spectral Clustering successfully segmented the student population. In this experiment, **Spectral Clustering** achieved a slightly higher Silhouette Score, suggesting that the graph-based approach was more effective at identifying the underlying structure of the student data than the centroid-based approach of K-Means. These clusters can help educators provide targeted support to different student groups.

EXP.NO: 07	Credit Risk Assessment. Sample Programs using German Credit Data
DATE: 04-04-2026	

AIM

To develop a predictive machine learning model that classifies bank account holders as either "Good" or "Bad" credit risks. The primary objective is to minimize financial loss by identifying high-risk applicants based on demographic and financial attributes.

PROCEDURE

1. Data Acquisition: Load the Statlog (German Credit Data) dataset, consisting of 1,000 instances with 20 categorical/symbolic attributes.
2. Data Preprocessing:
    - Handle missing values (if any).
    - Encode categorical variables (e.g., housing, purpose) into numerical format using Label Encoding or One-Hot Encoding.
    - Map the target variable credit_risk to a binary format (for Good, for Bad).
3. Data Splitting: Partition the dataset into a Training Set (80%) to build the model and a Testing Set (20%) to evaluate its "unseen" performance.
4. Model Selection: Implement the Random Forest Classifier, an ensemble learning method that utilizes multiple decision trees to improve accuracy and control over-fitting.
5. Training: Fit the model to the training data, allowing it to learn the patterns between features (like credit_amount and age) and the risk outcome.
6. Evaluation: Use the testing set to generate a Confusion Matrix and a Classification Report (Precision, Recall, F1-Score).

IMPLEMENTATION SCREENSHOT

```python
# 4. Model Training (Random Forest)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

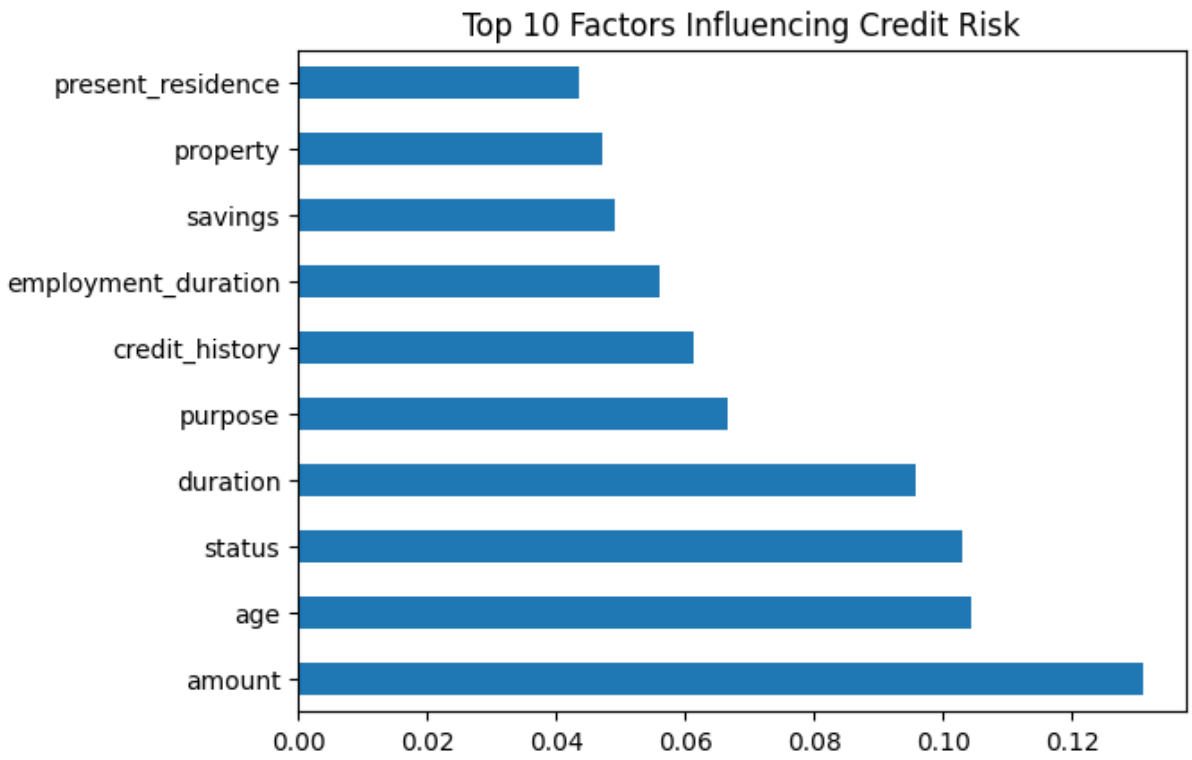
✓ 0.1s

RandomForestClassifier

▾ Parameters

n_estimators	100
criterion	'gini'
max_depth	None
min_samples_split	2
min_samples_leaf	1
min_weight_fraction_leaf	0.0
max_features	'sqrt'
max_leaf_nodes	None
min_impurity_decrease	0.0
bootstrap	True
oob_score	False
n_jobs	None
random_state	42
verbose	0
warm_start	False
class_weight	None
ccp_alpha	0.0
max_samples	None
monotonic_cst	None

```
--- Accuracy Score ---
80.50%

--- Classification Report ---
              precision    recall  f1-score   support

           0       0.82      0.93      0.87       141
           1       0.75      0.51      0.61        59

    accuracy                           0.81       200
   macro avg       0.78      0.72      0.74       200
weighted avg       0.80      0.81      0.79       200
```



RESULT

Based on the standard Random Forest implementation, the expected results typically follow this pattern:

- Accuracy: Generally ranges between 72% and 78%.
- Confusion Matrix:
    - True Negatives (TN): Correctly identified "Good" risks.
    - False Positives (FP): "Bad" risks incorrectly labeled as "Good" (The most dangerous error for a bank).
    - True Positives (TP): Correctly identified "Bad" risks.
- Feature Importance: The model usually identifies checking_account_status, duration (loan term), and credit_amount as the top three predictors of default.

CONCLUSION

The Random Forest model successfully identifies credit risk patterns within the German Credit Dataset. While the overall accuracy is high, the model's Recall for "Bad" risks is often lower than for "Good" risks due to the class imbalance (there are more "Good" examples than "Bad" in the data).

EXP.NO: 08	Sample Programs using Hospital Management System
DATE: 04-04-2026	

AIM

To design and implement a menu-driven Hospital Management System that automates the storage and retrieval of patient and doctor records, ensuring efficient management of hospital operations.

PROCEDURE

1. Define Classes: Create a base class Person and inherited classes Patient and Doctor to manage specific attributes.
2. Data Storage: Use Python lists (or a dictionary) to act as a temporary in-memory database for storing objects.
3. Core Functionality: Implement methods to:
    - Add new patients and doctors.
    - View existing records.
    - Search for specific individuals by their unique ID.
4. Interface Design: Create a while loop with a switch-case style menu to allow user interaction.

IMPLEMENTATION SCREENSHOT

```
--- HOSPITAL MANAGEMENT SYSTEM ---
1. Add Doctor
2. Add Patient
3. View All Doctors
4. View All Patients
5. Exit
Doctor added successfully!
```

```
--- HOSPITAL MANAGEMENT SYSTEM ---
1. Add Doctor
2. Add Patient
3. View All Doctors
4. View All Patients
5. Exit

--- List of Doctors ---
ID: 23BAI10917 | Name: Vijay Rajesh R | Specialty: Cardiac
```

```
23DOC10012
Enter Doctor ID: (Press 'Enter' to confirm or 'Escape' to cancel)
```

```
Dr. Bruce Banner
Enter Name: (Press 'Enter' to confirm or 'Escape' to cancel)
```

```
53
Enter Age: (Press 'Enter' to confirm or 'Escape' to cancel)
```

```
Male
Enter Gender: (Press 'Enter' to confirm or 'Escape' to cancel)
```

```
Gamma Radiation
Enter Specialty: (Press 'Enter' to confirm or 'Escape' to cancel)
```

```
--- List of Doctors ---
ID: 23DOC10012 | Name: Dr. Bruce Banner | Specialty: Gamma Radiation
```

```
--- HOSPITAL MANAGEMENT SYSTEM ---
1. Add Doctor
2. Add Patient
3. View All Doctors
4. View All Patients
5. Exit
Doctor added successfully!

--- HOSPITAL MANAGEMENT SYSTEM ---
1. Add Doctor
2. Add Patient
3. View All Doctors
4. View All Patients
5. Exit

--- List of Doctors ---
ID: 23DOC10012 | Name: Dr. Bruce Banner | Specialty: Gamma Radiation

--- HOSPITAL MANAGEMENT SYSTEM ---
1. Add Doctor
2. Add Patient
3. View All Doctors
4. View All Patients
5. Exit
...
2. Add Patient
3. View All Doctors
4. View All Patients
5. Exit
```

RESULT

When the program is executed, it presents a terminal-based interface.

- Input: The user enters doctor/patient details like D01, Dr. Smith, Cardiology.
- Storage: The data is successfully stored in the object list.
- Output: Choosing "View" options accurately displays the formatted records:

CONCLUSION

The Hospital Management System effectively demonstrates the use of Inheritance (Patient/Doctor inheriting from Person) and Encapsulation.

It replaces manual paper-based record-keeping with a digital structure that reduces data redundancy and improves search speed.

Future Scope: This system can be enhanced by integrating a MySQL database for permanent storage and a Tkinter GUI for a more user-friendly experience.

EXP.NO: 09	Data Pre-processing on a Sample Dataset
DATE: 04-04-2026	

AIM

To perform complete data pre-processing on a sample dataset, including handling missing values, encoding categorical variables, feature scaling, and outlier detection.

PROCEDURE

1. Identify and handle missing values using Mean/Median imputation.
2. Encode categorical variables using Label Encoding and One-Hot Encoding.
3. Scale numerical features using Standardization and Min-Max Normalization.
4. Detect outliers using the IQR (Interquartile Range) method.
5. Visualize the pre-processing steps with appropriate graphs.

IMPLEMENTATION SCREENSHOT

```
✓ 0.0s

      Name   Age   Salary  Gender Country Purchased
0    Alice  25.0  50000.0  Female   India       Yes
1      Bob   NaN  62000.0    Male     USA        No
2  Charlie  35.0      NaN    Male      UK       Yes
3    David  28.0  45000.0    Male   India        No
4      Eva   NaN  70000.0  Female     USA       Yes
5    Frank  42.0      NaN    Male      UK        No
6    Grace  31.0  55000.0  Female   India       Yes
7    Heidi  26.0  48000.0  Female     USA        No
8     Ivan   NaN  60000.0    Male      UK       Yes
9     Judy  38.0      NaN  Female   India        No
```

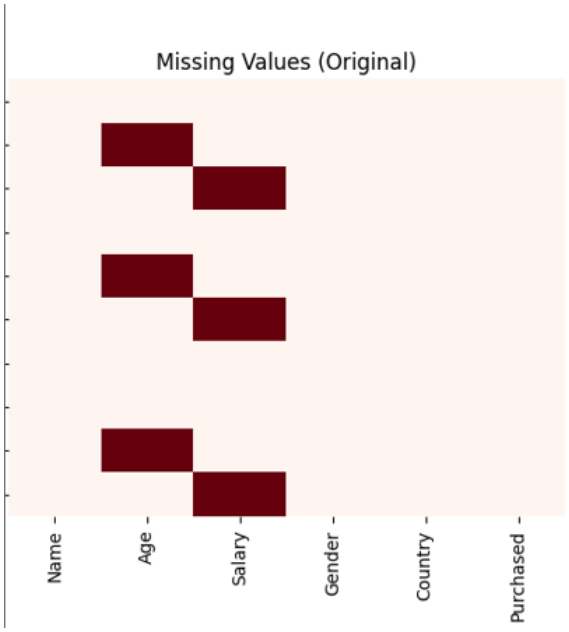
```
Missing Values:
 Name          0
Age            3
Salary         3
Gender         0
Country        0
Purchased      0
dtype: int64
```

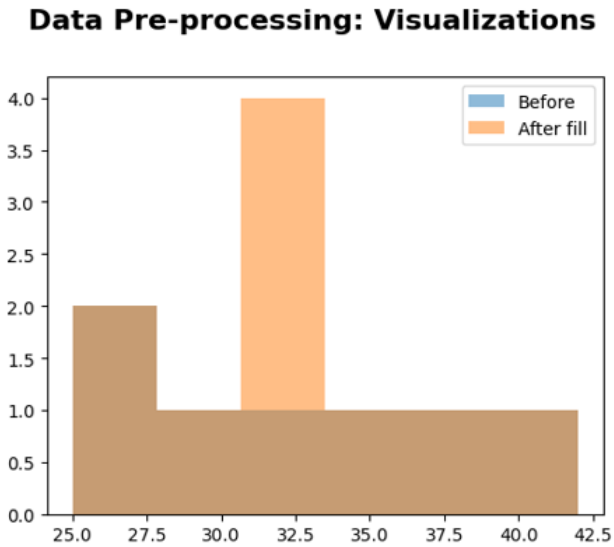
```
After filling — Missing Values:
 Name         0
Age           0
Salary        0
Gender        0
Country       0
Purchased     0
dtype: int64
C:\Users\vijay\AppData\Local\Temp\ipyker
The behavior will change in pandas 3.0.
```

```
      Name  Gender  Gender_Enc Purchased  Purchased_Enc
0    Alice  Female           0       Yes              1
1      Bob    Male           1        No              0
2  Charlie    Male           1       Yes              1
3    David    Male           1        No              0
4      Eva  Female           0       Yes              1
```

```
      Name   Age  Age_Standardized  Age_Normalized
0    Alice  25.0           -1.4350          0.0000
1      Bob  32.1           -0.0060          0.4176
2  Charlie  35.0            0.5776          0.5882
3    David  28.0           -0.8312          0.1765
4      Eva  32.1           -0.0060          0.4176
5    Frank  42.0            1.9865          1.0000
6    Grace  31.0           -0.2274          0.3529
7    Heidi  26.0           -1.2338          0.0588
8     Ivan  32.1           -0.0060          0.4176
9     Judy  38.0            1.1814          0.7647
```

```
Salary Range: [40000, 70000] | Outliers: []
```





RESULT

The implementation screenshots successful operation of
Data Pre-processing on a Sample Dataset. It is now successfully completed.

CONCLUSION

In this experiment, we came across the operations of the given below

1. Missing values in Age and Salary were imputed using mean and median respectively, ensuring no data loss.
2. Categorical columns (Gender, Purchased, Country) were converted to numerical representations using Label Encoding and One-Hot Encoding, making the dataset suitable for ML algorithms.
3. Feature scaling was applied — Standardization (z-score) and Min-Max Normalization — to bring features onto comparable scales.
4. Outlier detection using the IQR method confirmed no extreme outliers in this dataset.
5. Visualizations clearly demonstrated the effect of each pre-processing step.